

Introduction to Web Application Attacks

Unit 10 — How the Web Really Works

Almost everything you use online is a web app. Before you can find — or fix — a bug, you have to open the hood.

Learning objectives

By the end of this unit you can:

- **Describe** the HTTP request/response model and the roles of **client** and **server**.
- **Identify** the parts of a request (method, path, headers, body) and a response (status code, headers, body).
- **Break down** a URL into scheme, host, path, query string, and parameters.
- **Explain** the difference between **client-side** and **server-side**.
- **Use** View Source and Developer Tools to inspect HTML, cookies, and requests.
- **Configure** Burp Suite as an intercepting proxy and **intercept, read, and modify** a request.
- **Explain** the OWASP Top 10 and content discovery at an awareness level.

The web is a conversation

- A **client** (your browser) sends a **request**.
- A **server** receives it and sends back a **response**.
- The rules for that conversation are called **HTTP** (HTTPS = HTTP + encryption).

You can't attack — or defend — a web app you don't understand.

Anatomy of a request

```
GET /search?q=cats HTTP/1.1
Host: example.com
User-Agent: Firefox
Cookie: session=abc123
```

- **Method** — the verb: `GET` (fetch) or `POST` (send data)
- **Path** — what you're asking for (`/search`)
- **Headers** — extra info (`Host:`, `Cookie:` ...)
- **Body** — the content sent (form data, often empty on a GET)

Anatomy of a response

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Set-Cookie: session=abc123  
  
<html>...the page...</html>
```

- **Status code** — how it went (200 OK)
- **Headers** — info about the response
- **Body** — the actual HTML/content sent back

Status codes at a glance

Family	Meaning	Examples
2xx	Success	200 OK
3xx	Redirect	301 / 302
4xx	Client error	403 Forbidden, 404 Not Found
5xx	Server error	500

404 means the **client** asked for something that isn't there. 500 means the **server** broke.

Cookies, sessions, client vs server

- **Cookie** — a small piece of data the server tells your browser to store and send back.
- **Session** — how the server remembers *who you are* between requests.
- **Client-side** — runs in your browser (HTML/JS). **The user controls it.**
- **Server-side** — runs on the server (database, logic). The user can't see it.

Never trust the client. Anything client-side can be changed by the user. Real security must be enforced **server-side**.

Seeing what the server sent

- **View Source** — the raw HTML the server delivered (secrets sometimes hide in comments!).
- **Developer Tools** — inspect live:
 - **Elements** — the HTML tree
 - **Network** — every request/response
 - **Storage / Cookies** — what's stored in your browser

The OWASP Top 10 — the map

- **OWASP** = a nonprofit that publishes free web-security guidance.
- The **Top 10** = the ten most critical web app risks, including:
 - Broken Access Control
 - **Injection** (we attack this in Units 11–12)
 - Cryptographic Failures
 - Security Misconfiguration

Today this is the **map**. Units 11 and 12 do the driving.

Burp Suite: the intercepting proxy

- A **proxy** sits between your browser and the server.
- An **intercepting proxy** can **pause** a request so you can read or edit it.

```
Browser → [ BURP ] → Server  
           pause / read / edit
```

- Key tools: **Proxy** (intercept + HTTP history) and **Repeater** (resend & tweak one request).

Defense mindset: visibility is not security

- A proxy proves the user can **see and change** any request — even hidden fields and "validated" forms.
- **If your only check runs in the browser, it can be bypassed.**

Defense (server-side):

- Re-check and **validate every input on the server.**
- Mark session cookies `HttpOnly` and `Secure`.
- Never rely on hidden fields or JavaScript to enforce rules.

Ethics & Authorization

A proxy lets you **modify** real traffic — that makes the temptation bigger and the line clearer.

Targets are **intentionally vulnerable practice apps ONLY** — TryHackMe rooms, PortSwigger Academy, DVWA. **Never** your school portal, a bank, a game, or *any* real site — even just to "look."

Unauthorized testing is a crime under the **CFAA**. Authorization and scope are the only difference.

Key vocabulary

Term	Meaning
HTTP / HTTPS	Language of the web (HTTPS = encrypted)
Method / Status code	The request verb / the 3-digit result
Header / Body	Extra info / the main content
Query string / Parameter	After <code>?</code> / one <code>name=value</code> input
Cookie / Session	Stored data / how the server remembers you
Proxy / Intercepting proxy	Sits between / can pause & edit traffic
Repeater	Burp tool to resend & tweak one request
Content discovery	Guessing hidden paths from a wordlist



Lab launch

Platform: Burp Suite Community + OWASP Top 10 (TryHackMe)

- **Path A** — THM "Burp Suite: The Basics" AttackBox (all in-browser).
- **Path B** — local Burp → **Proxy** → **Intercept** → **Open Browser**.

You will: prove traffic flows through Burp → **intercept** a request → **modify** a parameter → send it to **Repeater** and compare responses.

Write your scope statement first: *"I am authorized to test only this intentionally vulnerable practice app."*

Content discovery (concept)

- Tools like **gobuster** / **dirb** request many guessed paths from a wordlist.

```
gobuster dir -u http://TARGET -w common.txt
```

- Read the **status codes** to find hidden pages:
 - **200** / **403** → something is **there**
 - **404** → nothing there

A **403 Forbidden** is *useful*: it means the path **exists** but is locked.

Recap

- The web is a **request → response** conversation over **HTTP**.
- A request has a **method, path, headers, body**; a response adds a **status code**.
- **Client-side is controllable** by the user — enforce security **server-side**.
- **Burp** lets you intercept, modify, and **Repeater** lets you resend & compare.
- The **OWASP Top 10** is the map for Units 11–12.

Exit ticket & discussion

1. Where does Burp sit in the conversation between your browser and the server?
2. Give one example of something that happens **client-side** and one **server-side**.
3. **Discuss:** Why is "I only looked, I changed nothing" still a problem when the target is a real site?

Next — Unit 11: Common Web Attacks: XSS, Command Injection & File Inclusion